

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 10%

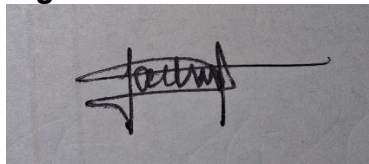
QUESTIONS		
Q.No	Question	Bloom's Level
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand
9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply

10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply
----	--	-------------

Code of Conduct:

I J a s h w a n t h . G . R -192524384 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. Correct the CREATE TABLE statement

Incorrect

```
CREATE TABLE Student  
(  
StudentID INT,  
Name VARCHAR(50),  
Age INT  
);
```

Correct

```
CREATE TABLE Student  
(  
StudentID INT PRIMARY KEY,  
Name VARCHAR(50) NOT NULL,  
Age INT CHECK(Age>=18)  
);
```

Corrections

- Added PRIMARY KEY
- Added NOT NULL
- Added CHECK constraint

2. Aggregate Function without GROUP BY

Incorrect

```
SELECT DeptID, COUNT(*)  
FROM Employee;
```

Correct

```
SELECT DeptID, COUNT(*)  
FROM Employee  
GROUP BY DeptID;
```

Reason: Every non-aggregate column (DeptID) must appear in the GROUP BY clause.

3. Misspelled Column Name

Incorrect

```
SELECT StudntName  
FROM Student;
```

Correct

```
SELECT StudentName  
FROM Student;
```

Reason: Correct the spelling of the column name.

4. JOIN Condition Missing

Incorrect

```
SELECT *  
FROM Employee  
JOIN Department;
```

Correct

```
SELECT *  
FROM Employee e  
JOIN Department d  
ON e.DeptID = d.DeptID;
```

Reason: Every JOIN requires a valid join condition.

5. Subquery Returns Multiple Values

Incorrect

```
SELECT *  
FROM Employee  
WHERE DeptID =  
(  
SELECT DeptID
```

```
FROM Department
```

```
);
```

Correct

```
SELECT *
```

```
FROM Employee
```

```
WHERE DeptID IN
```

```
(
```

```
SELECT DeptID
```

```
FROM Department
```

```
);
```

Reason: Use IN when the subquery returns multiple rows.

6. Debug VIEW Creation

Incorrect

```
CREATE VIEW Student_View AS
```

```
SELECT StudentID, StudentAge
```

```
FROM Student;
```

Correct

```
CREATE VIEW Student_View AS
```

```
SELECT StudentID, Age
```

```
FROM Student;
```

Reason: StudentAge does not exist; the correct column is Age.

7. Optimize WHERE Clause

Inefficient

```
SELECT *
```

```
FROM Employee
```

```
WHERE Salary>50000
```

AND Salary>30000;

Optimized

SELECT *

FROM Employee

WHERE Salary>50000;

Reason: Salary > 30000 is redundant.

8. Integrity Constraint Violations

Incorrect

EmployeeID = NULL

Salary = -5000

Correct Table

CREATE TABLE Employee

(

EmployeeID INT PRIMARY KEY,

Name VARCHAR(50) NOT NULL,

Salary DECIMAL CHECK(Salary>0)

);

Reason

- Primary key cannot be NULL.
- Salary should be greater than zero.

9. Remove Repeating Groups (1NF/2NF)

Unnormalized Table

StudentID Name Subjects

101	Rahul	DBMS, Java, Python
-----	-------	--------------------

1NF

StudentID Name Subject

101	Rahul	DBMS
101	Rahul	Java
101	Rahul	Python

2NF

Student

StudentID Name

101	Rahul
-----	-------

Enrollment

StudentID Subject

101	DBMS
101	Java
101	Python

10. Remove Redundant Attributes (3NF)

Original Relation

| StudentID | StudentName | DeptID | DeptName |

Functional Dependency

DeptID → DeptName

3NF Decomposition

Student

| StudentID | StudentName | DeptID |

Department

| DeptID | DeptName |

Reason

- Removes redundancy.
- Eliminates update, insertion, and deletion anomalies.
- Achieves Third Normal Form (3NF).